



Rueda de Platos

Documentación técnica y funcional de la aplicación

App web personal (PWA) para generar menús semanales variados a partir de tus propias comidas, sin recomendaciones externas, con control de frecuencias por alimento y cálculo automático de objetivos nutricionales.

Preparado para retomar el desarrollo en Claude Code · Agosto 2026

1. Qué es la aplicación y por qué existe

Rueda de Platos es una aplicación web de uso estrictamente personal (PWA — Progressive Web App), pensada para resolver un problema muy concreto: dejar de tener que pensar qué comer en cada comida, sin que ninguna app decida por ti *qué* comer. La idea de fondo es que tú aportas tus propios platos íntegros (los que ya sueles cocinar), y la aplicación se limita a combinarlos en un menú semanal variado, respetando reglas de frecuencia que tú mismo defines.

Los dos objetivos que guían todo el diseño son:

- **Variar** las comidas semana a semana sin esfuerzo mental, evitando caer siempre en lo mismo.
- **Evitar atracones** — dar cierta estructura y previsibilidad a la alimentación, sin llegar a un control obsesivo de cada gramo, para tener una relación más tranquila con la comida.

Un principio importante que se ha mantenido en todo el desarrollo: la app **no elige qué es un plato** ni lo desglosa por ti — tú decides que "Pollo + Arroz + Ensalada" es una comida válida, con qué frecuencia quieres que aparezca, y la app solo organiza la variedad y las cantidades dentro de esa estructura que ya has definido.

2. Arquitectura técnica

La app está montada sin proceso de compilación (build): es HTML + JSX que el propio navegador transforma al vuelo con Babel Standalone, cargado por *import maps* directamente desde CDN (esm.sh). Esto la hace muy fácil de editar y desplegar (basta con subir los archivos a GitHub Pages), a cambio de una carga inicial algo más lenta que con un build optimizado.

Archivo	Función
index.html	Punto de entrada: carga el import map, Babel, registra el service worker y arranca auth-bootstrap.jsx.
auth-bootstrap.jsx	Login con Firebase Auth (email/contraseña y Google) y App Check. Una vez autenticado, monta app.jsx.
app.jsx	El núcleo de la aplicación: catálogo de alimentos e ingredientes, motor de generación de menús, cálculo de objetivos, y todas las pantallas (~3.700 líneas).
manifest.json	Metadatos de PWA: nombre, icono, colores — permite "Añadir a pantalla de inicio".
sw.js	Service worker: cachea los archivos con estrategia "red primero" para que funcione offline y se actualice solo al haber conexión.
icon.svg	Icono de la app.

Guardado de datos: Firebase Firestore, ligado a tu cuenta de usuario (login con email o Google). Todo lo que configures (ingredientes, frecuencias, perfil, historial de menús) se guarda en la nube asociado a tu usuario, no en el propio navegador — así no se pierde nada al cambiar de dispositivo.

3. Cómo se organizan los ingredientes

Cada ingrediente que das de alta pertenece a una **categoría** y tiene un **tipo de regla** que decide cómo de a menudo aparece en el menú. Esta combinación es el corazón de todo el sistema de variedad.

3.1 Categorías

Categoría	Qué es	Ejemplos del catálogo actual
proteína	El componente principal de comida/cena.	Pollo, salmón, merluza, atún, sardinas, carne picada, hamburguesas
carbo	La base de hidratos de la comida.	Pasta, arroz, gnocchi, patatas
verdura	El acompañamiento vegetal.	Ensalada completa, ensalada de tomate, pimiento y cebolla, purés
grasa	Grasas de ajuste (aceite, frutos secos, guacamole...).	Aceite de oliva, pistachos, anacardos, guacamole
desayuno / merienda	Combinaciones fijas para esas dos franjas.	(pendiente de rellenar por ti, según lo hablado)
especial	Modificadores que se añaden sobre una comida ya elegida.	Garbanzos (se suma a la ración de carbo)
cerrado	Platos ya completos, cerrados, que sustituyen a la estructura proteína+carbo+verdura.	Hamburguesa completa con pan, nachos, huevos rotos

3.2 Tipos de regla (cómo se decide la frecuencia)

Tipo de regla	Cómo funciona
base	El "relleno por defecto": aparece en todos los huecos que no ocupa ninguna otra regla. Es el caso de Pollo (carne) — es intencionadamente el ingrediente más repetido, en torno a la mitad de las comidas/cenas del ciclo.
frecuencia	Aparece un número fijo de veces por período (semana o ciclo completo de 2 semanas). Es el mecanismo pensado para "esto no puede faltar, pero tampoco saturar" — por ejemplo el pescado.
probabilidad	Compite por un peso relativo (%) frente a otros ingredientes de su misma categoría. Se usa para elementos intercambiables entre sí, como los carbohidratos base (pasta/arroz/gnocchi/patatas) o las verduras.
bloque_miembro	Varios ingredientes agrupados en un "bloque": el bloque tiene su propia frecuencia total, y dentro se reparte entre sus miembros según su peso relativo. Así, "carne roja" reparte entre picada de cerdo/vacuno y hamburguesa de ternera/cerdo sin que compitan con el resto del catálogo.

3.3 Frecuencias actualmente configuradas (proteínas)

Proteína	Frecuencia
Salmón	1 vez por semana
Merluza	1 vez por semana
Atún	2 veces por semana
Sardinas	1 vez por ciclo completo (2 semanas)
Pollo (carne)	Base — rellena el resto de huecos
Carne roja (picada cerdo/vacuno + hamburguesa ternera/cerdo)	Bloque con frecuencia propia, repartida entre sus 2 miembros por peso

Proteína	Frecuencia
Pollo picado (carne picada de pollo + hamburguesa de pollo)	Bloque con frecuencia propia; dentro del bloque, la picada pesa 3 veces más que la hamburguesa
Garbanzos (especial)	2 veces por semana — se añade como modificador de carbohidrato en la comida que le toque

Estas frecuencias son completamente editables desde la propia app (pestaña Configuración → Alimentos), sin tocar código.

4. El motor de generación del menú

Cuando pulsas "Generar menú", la app construye un ciclo de 2 semanas (14 días × 2 comidas principales = 28 huecos de comida/cena, más desayunos y meriendas aparte) siguiendo este proceso:

- **Reparto de huecos por frecuencia:** primero se calcula cuántas veces debe aparecer cada ingrediente de tipo "frecuencia" o "bloque" en el ciclo, y se reservan esos huecos concretos (método del resto mayor, para repartir de forma justa incluso cuando los números no encajan exactos).
- **Relleno con la base:** los huecos que sobran se rellenan con el ingrediente "base" (Pollo).
- **Elección de carbohidrato y verdura** para cada comida, por sorteo ponderado según las probabilidades que hayas configurado.
- **Reglas de combinación (afinidad):** puedes definir reglas del tipo "si sale Salmón, sube la probabilidad de que la verdura sea Ensalada completa" (niveles: nula/baja/alta/máxima). Estas reglas solo se evalúan cuando el resto de elementos de esa combinación ya están decididos.
- **Anti-repetición:** el generador recuerda la última elección de cada grupo de un solo hueco (por historial) para evitar repetir exactamente lo mismo la semana siguiente cuando hay alternativa disponible.
- **Ajuste de raciones a tus objetivos:** si tienes un perfil con objetivos calculados, la proteína y el carbohidrato de cada comida se ajustan para acercarse a su parte proporcional del objetivo; la grasa queda como resultado, con el aceite de oliva como "mando de ajuste fino" visible.

Es importante remarcar la filosofía detrás del punto 6: no existe una única combinación de gramos que cuadre exactamente los 4 objetivos (kcal, proteína, grasa, carbohidrato) cada día — matemáticamente hay infinitas soluciones posibles, y further, algunos días el menú no tiene ningún alimento con mucha grasa, así que no se puede inventar de la nada. El objetivo realista es acercarse mucho, con la **media semanal** como referencia más fiable que un solo día.

5. Base de datos de alimentos y cálculo de macros

Cada ingrediente del menú está enlazado a un alimento de un catálogo interno con sus macros por 100g (actualmente unos 30 alimentos, con calorías, proteína, grasa, carbohidratos, y en muchos casos también sal, azúcares, fibra y grasa saturada). El origen del dato queda marcado como "Etiqueta" (sacado del envase real del producto que usas) o "Estándar" (valor de referencia contrastado, para alimentos genéricos sin marca fija, como el tomate o la patata).

Con esto, cada comida generada calcula sus macros exactos escalando por la cantidad real de gramos, y esos totales se agregan por comida, por día y por semana — es lo que alimenta las gráficas de calorías y la "rueda" de macros de la app, y lo que hace posible comparar el menú generado contra tus objetivos.

6. Perfil y cálculo de objetivos nutricionales

La primera vez que entras sin perfil guardado, la app te pide (una sola vez) nombre, sexo, año de nacimiento, altura, peso y nivel de actividad. A partir de ahí, estos datos solo se tocan desde la pestaña "Perfil", y cada vez que los guardas, los objetivos se recalculan automáticamente.

6.1 Fórmula actual: Mifflin-St Jeor

Es la fórmula que la literatura científica considera más fiable para población general (más precisa que la clásica Harris-Benedict), con un margen de error típico de $\pm 10\%$ frente a medición real en laboratorio.

$$\text{BMR} = 10 \times \text{peso(kg)} + 6,25 \times \text{altura(cm)} - 5 \times \text{edad} + 5 \text{ (hombre)} / - 161 \text{ (mujer)}$$

$$\text{TDEE (mantenimiento)} = \text{BMR} \times \text{factor de actividad}$$

6.2 Tabla de niveles de actividad (valor actual, un único multiplicador por nivel)

Nivel	Descripción	Factor Mifflin	Proteína g/kg	Grasa g/kg
Sedentario	Sin ejercicio, <5.000 pasos/día	1,20	1,0	0,85
Ligero	1-2 días/semana	1,375	1,3	0,9
Moderado	3-5 días/semana	1,55	1,7	1,0
Alto	6-7 días/semana	1,725	2,0	1,25

Los carbohidratos no tienen un g/kg propio: se calculan como lo que resta del total de calorías una vez descontadas las kcal de proteína ($\times 4$) y grasa ($\times 9$). Los gramos/kg de cada nivel son el punto medio de los rangos seguros encontrados en la literatura (proteína 15-25% de la energía, grasa 25-35%, carbohidratos 40-55%).

6.3 Reparto del objetivo diario entre las 4 comidas

Desayuno	Comida	Merienda	Cena
20%	35%	15%	30%

6.4 Ejemplo real verificado (perfil: hombre, 2005, 170cm, 70,5kg, actividad moderada)

Métrica	Valor calculado
BMR	1.667,5 kcal
TDEE (mantenimiento)	≈ 2.585 kcal
Proteína objetivo	≈ 120 g
Grasa objetivo	≈ 70 g
Carbohidratos objetivo	≈ 368 g

Al comparar contra un menú real generado por la app, la media semanal de calorías (~ 2.559 kcal) coincidió casi exactamente con este objetivo (diferencia $\sim 1\%$) — confirmando que el motor de cálculo funciona como se diseñó. Donde sí se detectó margen de mejora fue en el reparto entre macros dentro de ese total (algo más de proteína y algo menos de carbohidrato de lo estrictamente calculado), algo a vigilar en el motor de ajuste de raciones.

7. Próximo cambio planteado: nivel de actividad más preciso

Motivo del cambio. El sistema actual de 4 niveles fijos tiene una limitación conocida: el factor de actividad (PAL) está pensado para clasificar el estilo de vida *completo* de una persona (trabajo, movimiento diario, tiempo libre), no solo sus días de entrenamiento. Esto hace que alguien con un día a día sedentario (estudios o trabajo de escritorio) que entrena 3-5 días por semana quede sobreestimado si se clasifica sin más como "moderado", porque ese factor asume que el resto del día también es más activo de lo que realmente es.

7.1 De dónde viene la evidencia

Los factores PAL (Physical Activity Level = TDEE/BMR) provienen del informe conjunto FAO/OMS/UNU (2001), construido a partir de estudios con agua doblemente marcada — el método más fiable que existe hoy para medir gasto energético real en condiciones de vida libre (no en laboratorio estático). El problema de aplicarlos tal cual en una app es que comprimen todo el estilo de vida en una única cifra elegida por "días de entrenamiento", perdiendo la diferencia entre "vida sedentaria + entrena fuerte" y "vida activa + entrena fuerte", que en la práctica dan un gasto energético total distinto.

7.2 El enfoque propuesto: separar vida diaria y entrenamiento

En vez de un único desplegable de actividad, calcular el TDEE en dos partes independientes:

$$\text{TDEE} = (\text{BMR} \times \text{PAL_base del día a día}) + (\text{kcal específicas del entrenamiento})$$

- **PAL_base** (solo tu vida sin contar el entrenamiento): trabajo/estudios de escritorio $\approx 1,2$ - $1,3$; de pie/moviéndote bastante $\approx 1,4$ - $1,5$; físicamente exigente $\approx 1,6$ +
- **Kcal del entrenamiento**, calculadas aparte con METs (equivalentes metabólicos, estandarizados en el *Compendium of Physical Activities*), con la fórmula: $\text{MET} \times \text{peso(kg)} \times \text{horas de entrenamiento}$. Por ejemplo, pesas ronda los 6 METs; correr a ritmo medio, 8-10 METs.

7.3 Ejemplo con el mismo perfil (70,5kg, día a día sedentario + 4 sesiones de pesas de 1h/semana)

Componente	Cálculo	Resultado
PAL_base (vida sedentaria)	$1.667,5 \times 1,25$	$\approx 2.084 \text{ kcal}$
+ Entrenamiento	$6 \text{ METs} \times 70,5\text{kg} \times 1\text{h} \times 4 \text{ sesiones/semana} \div 7$	$\approx +242 \text{ kcal/día de media}$
TDEE afinado		$\approx 2.326 \text{ kcal/día}$

Este resultado cae entre el nivel "ligero" (2.293 kcal) y "moderado" (2.585 kcal) del sistema actual — justo lo que cabía esperar, porque ese es exactamente el matiz que el selector de 4 niveles no puede capturar por sí solo.

7.4 Qué implica el cambio a nivel de app

- El perfil (onboarding y pestaña "Perfil") pasaría de un único selector de actividad a: tipo de día a día (3-4 opciones de PAL_base) + una lista de entrenamientos habituales (tipo, duración, frecuencia semanal), de la que se derivan las kcal de entrenamiento.
- `calcularObjetivosPerfil()` cambiaría su fórmula de TDEE, manteniendo intactos los g/kg de proteína y grasa por nivel de actividad general (o adaptándolos también si se decide en su momento).
- El resto del sistema (reparto 20/35/15/30, motor de ajuste de raciones, exportación) no necesita cambios, porque todos consumen el objetivo diario ya calculado, sin depender de cómo se calculó.

Aviso honesto que se mantiene: ni este método más afinado deja de ser una estimación (margen de error habitual $\pm 10\text{-}15\%$). El paso final de calibración sigue siendo pesarse 1-2 veces/semana durante 2-3 semanas comiendo al objetivo calculado, y ajustar un 5-10% hacia arriba o abajo según la tendencia real del peso — eso corrige cualquier fórmula mejor que cualquier fórmula por sí sola.

Fin de la documentación. Generado a partir del código fuente actual de la app (app.jsx, auth-bootstrap.jsx, index.html, manifest.json, sw.js) y del histórico de decisiones tomadas en las conversaciones previas de diseño.